

1. 内存分配方式及其区别

内存分配方式主要有以下几种：

- **静态内存分配**：在编译时确定内存的大小和位置。常见于全局变量、静态变量和常量。这些变量的生命周期是整个程序运行期间。
- **自动内存分配（栈内存分配）**：在函数调用时分配，函数结束时自动释放。适用于局部变量和函数参数。栈内存分配速度较快，但由于栈的大小有限，容易导致栈溢出。
- **动态内存分配**：在运行时通过函数如 `malloc`、`calloc`、`realloc` 和 `free` 在堆上分配和释放内存。动态内存分配允许在程序运行时根据需要调整内存使用，但可能导致内存泄漏和碎片化。
- **内存映射**：使用 `mmap` 函数将文件或设备映射到内存中，适用于大文件的访问或共享内存。映射区域可以被读取、写入或执行。

2. `const char *p` 和 `char * const p` 的区别

- `const char *p`：`p` 是一个指向 `const char` 的指针，意味着 `p` 所指向的数据不能被修改，但指针本身可以指向其他 `char` 类型的地址。
- `char * const p`：`p` 是一个 `const` 指针，意味着指针 `p` 不能被修改（即，不能指向其他地址），但指针指向的数据可以被修改。

3. `extern "C"` 的作用

在 C++ 中，`extern "C"` 用于告诉编译器以 C 语言的方式来处理指定的函数或变量，主要用于实现 C++ 程序调用用 C 编译器编译的函数。这样做的原因是 C++ 支持函数重载，而 C 语言不支持。使用 `extern "C"` 可以避免由于 C++ 的名称修饰（name mangling）导致的链接错误。

4. 逆序链表

```
void reverseList(Node*& head) {
    Node* prev = nullptr;
    Node* curr = head;
    Node* next = nullptr;

    while (curr != nullptr) {
        next = curr->next; // 保存下一个节点
        curr->next = prev; // 反转当前节点的指针
        prev = curr;      // 移动 prev 和 curr 指针
        curr = next;
    }
    head = prev; // 更新头节点
}
```

5. 程序输出和内存分布

程序1:

```
#include <iostream>
using namespace std;

class A {
```

```

public:
    virtual void aa(){}
private:
    char k[3];
};

class B: public A {
public:
    virtual void bb(){}
};

int main() {
    cout<<"A's size is "<<sizeof(A)<<endl;
    cout<<"B's size is "<<sizeof(B)<<endl;
    return 0;
}

```

执行结果:

```

A's size is 8
B's size is 8

```

内存分布:

- 类 A 包含一个虚函数表指针和一个 3 字节的 char 数组。通常情况下，虚函数表指针占用 4 字节（在 32 位系统上），加上 3 字节数组，整体对齐到 8 字节。
- 类 B 继承自 A，除了 A 的虚函数表指针和内存布局，B 还包含一个虚函数表指针，因此 sizeof(B) 和 sizeof(A) 相同。

程序2:

```

#include <iostream>
using namespace std;

class A {
public:
    virtual void aa(){}
    virtual void aa2(){}
private:
    char k[3];
};

class B {
public:
    virtual void bb(){}
    virtual void bb2(){}
};

class C: public A, public B {
public:
    virtual void aa(){} //重写了A的aa()
    virtual void cc(){}
};

```

```
int main() {
    cout<<"A's size is "<<sizeof(A)<<endl;
    cout<<"B's size is "<<sizeof(B)<<endl;
    cout<<"C's size is "<<sizeof(C)<<endl;
    return 0;
}
```

执行结果:

```
A's size is 12
B's size is 8
C's size is 16
```

内存分布:

- 类 **A** 包含两个虚函数表指针（每个 4 字节），以及 3 字节的 `char` 数组，通常对齐到 12 字节。
- 类 **B** 包含两个虚函数表指针，通常对齐到 8 字节。
- 类 **C** 继承自 **A** 和 **B**，因此需要存储两个虚函数表指针，以及 **A** 和 **B** 的内存布局，加上 `cc` 方法所需的内存，总共 16 字节。

6. 继承控制

为了确保子类不能修改基类的某些成员变量，可以将这些变量声明为 `protected` 或 `private`。同时，继承方式应该是 `protected` 或 `private`。`public` 继承方式会暴露基类的公共和保护成员变量。

7. 引用与指针的区别

- **引用**：必须在声明时初始化，不能改变引用的对象。引用通常用于函数参数传递和返回值，可以提高效率并避免复制。
- **指针**：可以在运行时改变指向的对象，可以为空。指针允许更多的灵活性，但需要显式地管理内存和处理空指针。

8. 引用作为函数返回值

格式:

```
int& foo() {
    static int value = 42;
    return value;
}
```

好处:

- 避免了值复制的开销，直接返回对象的引用。
- 允许在函数外部直接修改返回的对象。

规则:

- 返回局部变量的引用是不安全的，因为函数结束后局部变量会被销毁。
- 引用必须引用一个有效的对象。

9. static 和 const 关键字的作用

- `static`:
 - **在函数内**: 将变量的生命周期扩展到整个程序运行期间, 保持其值。
 - **在类内**: 声明类的静态成员, 所有对象共享。
 - **在文件内**: 限制符号的链接范围, 仅在当前文件有效。
- `const`:
 - **常量数据**: 确保变量的值在初始化后不能修改。
 - **常量指针**: 指针指向的数据不能被修改。
 - **常量成员函数**: 保证成员函数不会修改对象的状态。

10. C 和 C++ 中 struct 和 class 的区别

- `struct`:
 - 默认访问权限是 `public`。
 - 默认继承方式是 `public`。
- `class`:
 - 默认访问权限是 `private`。
 - 默认继承方式是 `private`。

11. 内联函数和宏的差别

- **内联函数**:
 - 由编译器处理, 避免了函数调用的开销。
 - 支持类型检查和调试。
- **宏**:
 - 由预处理器处理, 直接进行文本替换。
 - 不支持类型检查, 可能导致错误和调试困难。

12. 反向遍历 vector 的错误

```
vector<int> array;
array.push_back(1);
array.push_back(2);
array.push_back(3);
for (vector<int>::size_type i = array.size() - 1; i >= 0; --i) {
    cout << array[i] << endl;
}
```

错误: `i` 的类型是 `vector<int>::size_type`, 它是无符号类型。`i` 递减到 0 后, 再减去 1 会变成最大值, 从而导致无限循环。应该使用带符号整数类型, 例如 `int`。

13. 面向对象的三个基本特征

- **封装**：将数据和操作数据的方法封装在一个类中，隐藏内部实现细节，对外提供接口。
- **继承**：允许创建新类基于已有类，从而复用已有的代码和功能。
- **多态**：通过相同的接口调用不同的实现，允许在运行时决定使用哪个方法。

14. 32 位 ARM 系统下结构体 `sizeof(A)`

```
struct A {  
    char t:4;  
    char k:4;  
    unsigned short i:8;  
    unsigned long m;  
};
```

- `char` 位域占用 1 字节 (8 位)，`unsigned short` 位域占用 1 字节 (8 位)，`unsigned long` 通常占用 4 字节 (32 位)。
- 对齐要求：`unsigned long` 需要对齐到 4 字节。

`sizeof(A)` = 12 字节 (8 字节用于位域 + 4 字节对齐)。

15. 程序的错误或效果

```
#define MAX 255  
int main() {  
    unsigned char A[MAX], i;  
    for (i = 0; i <= MAX; i++)  
        A[i] = i;  
}
```

错误：`i <= MAX` 的循环条件导致 `i` 超出 `unsigned char` 的范围，可能引发数组越界访问。`MAX` 为 255，但 `i` 从 0 到 255，总共 256 次迭代，会越界访问数组 `A` 的第 256 个元素。

16. 程序的错误或效果

```
void GetMemory(char **p, int num) {  
    *p = (char *)malloc(num);  
}  
void Test(void) {  
    char *str = NULL;  
    GetMemory(&str, 100);  
    strcpy(str, "hello");  
    printf(str);  
}
```

错误：

- `malloc` 的返回值未进行检查，可能会导致空指针解引用。
- `printf` 中使用了未格式化的字符串，应该使用 `printf("%s", str);`。

17. sizeof 计算

```
char str[] = "raysharp.cn";
char *p = str;
int n = 10;
```

- `sizeof(str)`: `str` 是一个数组，大小是字符串长度加上终止字符。"raysharp.cn" 的长度是 11 字节，所以 `sizeof(str) = 12`。
- `sizeof(p)`: `p` 是一个指针，在 32 位系统下，`sizeof(p) = 4` 字节。
- `sizeof(n)`: `n` 是一个 `int`，在 32 位系统下，`sizeof(n) = 4` 字节。

```
void Foo(char str[100]) {
    // 这里 str 是一个指针，不是数组
}
```

- `sizeof(str)`: 在函数参数中，`str` 被视为指针，`sizeof(str) = 4` 字节。

```
void *p = malloc(100);
```

- `sizeof(p)`: `p` 是一个指针，`sizeof(p) = 4` 字节。

18. 复制视频文件的函数

```
#include <fstream>

void copyFile(const std::string& sourcePath, const std::string& destPath) {
    std::ifstream src(sourcePath, std::ios::binary);
    std::ofstream dest(destPath, std::ios::binary);

    dest << src.rdbuf();
}
```

19. 交叉编译器的作用

交叉编译器用于在一种平台上编译代码，以便在另一种平台上运行。例如，在 Windows 上编译代码以在嵌入式 ARM 设备上运行。它允许开发者为目标平台生成可执行代码，而无需在目标平台上进行开发和测试。

20. Shell 脚本判断程序

```
#!/bin/bash
a=0
if [ $a -eq 0 ]; then
    echo "correct"
else
    echo "incorrect"
fi
```

21. 查找并删除 `.o` 文件

```
find . -type f -name "*.o" -delete
```

22. 判断表达式正确性

```
int a = 4;
(A) a += (a++); // a = 4 + 4; a = 8
(B) a += (++a); // a = 8 + 6; a = 14
(C) (a++) += a; // 错误, 不能将右值赋值给左值
(D) (++a) += (a++); // a = 5; a = 5 + 6; a = 11
```

23. 字符设备与块设备的区别

- **字符设备**: 数据按字符流的方式传输。没有固定的大小和结构。例子: 串口设备、终端。
- **块设备**: 数据以块的形式进行传输, 每个块具有固定的大小。适用于需要随机访问的存储设备。例子: 硬盘、USB 驱动器。

好的, 我们来逐一完成这些试题:

1. 不能做 `switch()` 的参数类型是?

`switch()` 语句的参数类型必须是整型数据 (`int`、`char`、`enum` 等), 不能是浮点型 (`float`、`double`)、指针或其他类型。具体来说, 以下类型不能用作 `switch()` 的参数:

- `float`
- `double`
- `char*` (指针类型)
- `void*` (空指针类型)

2. C++ 中如何声明 `const void f(void)` 函数为 C 程序中的库函数?

要在 C++ 中声明一个函数为 C 语言中的库函数, 需要使用 `extern "C"` 来防止 C++ 编译器对函数名进行名称修饰。声明 `const void f(void)` 为 C 语言中的库函数可以如下实现:

```
extern "C" void f(void);
```

3. `const char *p` 和 `char * const p` 的区别

- `const char *p`: `p` 是一个指向 `const char` 的指针, 意味着 `p` 所指向的数据不能被修改, 但指针本身可以指向其他 `char` 类型的地址。
- `char * const p`: `p` 是一个 `const` 指针, 意味着指针 `p` 不能改变它指向的地址, 但指针指向的数据可以被修改。

4. 面向对象程序设计的优点

- **封装**：将数据和操作数据的方法封装在一个类中，隐藏内部实现细节，对外提供接口。
- **继承**：允许创建新类基于已有类，从而复用已有的代码和功能。
- **多态**：通过相同的接口调用不同的实现，允许在运行时决定使用哪个方法。

5. 举例结构体中的一个 `char` 变量怎么拆分成两个变量

假设结构体中有一个 `char` 变量 `c`，我们可以将其拆分成两个 `char` 变量，如下：

```
struct Example {  
    char c;  
};  
  
void splitChar(Example* e, char* high, char* low) {  
    *high = (e->c >> 4) & 0x0F; // 高四位  
    *low = e->c & 0x0F; // 低四位  
}
```

6. BOOL, int, float, 指针类型的变量 `a` 与“零”的比较语句

- **BOOL**:

```
BOOL a;  
if (a == 0) { /* ... */ }
```

- **int**:

```
int a;  
if (a == 0) { /* ... */ }
```

- **float**:

```
float a;  
if (a == 0.0f) { /* ... */ }
```

- **指针**:

```
void *a;  
if (a == NULL) { /* ... */ }
```

7. 为 `Type *s` 在堆中分配空间

```
Type *s = (Type *)malloc(sizeof(Type));
```

8. 什么是动态特性？

动态特性通常指程序在运行时的行为特征。例如，在面向对象编程中，动态特性包括多态、动态绑定等，这允许在运行时根据实际对象类型来决定方法的调用。

9. 全局变量可不可以定义在可被多个 `.c` 文件包含的头文件中？为什么？

不建议将全局变量定义在可被多个 `.c` 文件包含的头文件中。因为这样会导致每个包含该头文件的 `.c` 文件都有一个全局变量的副本，从而引发链接错误。全局变量应在一个 `.c` 文件中定义，在头文件中只声明（`extern`）。

10. 已知二叉树后序遍历序列是 `dabec`，中序遍历序列是 `debac`，它的前序遍历序列是？

根据给定的后序和中序遍历序列，我们可以推断前序遍历序列。构造树的过程如下：

- 后序遍历的最后一个字符是树的根：`c`。
- 在中序遍历中，`c` 的左子树是 `debac` 中的 `deba`，右子树为空。
- 对 `deba` 进行相同的分析。

最终的前序遍历序列是：`c + dab + e`，即 `cdeb`。

11. 进程间通信的方式有哪些？

- **管道** (Pipes)：用于同一进程组或父子进程之间的通信。
- **命名管道** (Named Pipes)：用于不同进程之间的通信。
- **消息队列** (Message Queues)：允许进程之间以消息的形式交换数据。
- **共享内存** (Shared Memory)：多个进程可以访问同一块内存区域。
- **信号量** (Semaphores)：用于进程间的同步。
- **套接字** (Sockets)：可以用于不同计算机间的通信，支持网络通信。

12. 输出结果

```
#include <stdio.h>
union {
    int i;
    int j;
} a;

void main() {
    a.i = 5;
    a.j = 6;
    printf("%d", a.i);
}
```

输出结果是 `6`。由于 `union` 中的所有成员共享同一块内存，所以最后对 `a.j` 的赋值会覆盖 `a.i` 的值。

13. 实现数组的降序排序

```
#include <stdio.h>

void sort(int arr[], int size) {
    int temp;
    for (int i = 0; i < size - 1; i++) {
        for (int j = i + 1; j < size; j++) {
            if (arr[i] < arr[j]) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}

int main() {
    int array[] = {45, 56, 76, 234, 1, 34, 23, 2, 3};
    int size = sizeof(array) / sizeof(array[0]);
    sort(array, size);

    for (int i = 0; i < size; i++) {
        printf("%d ", array[i]);
    }
    return 0;
}
```

14. 程序问题

```
void GetMemory(char **p, int num) {
    *p = (char *)malloc(num);
}

void Test(void) {
    char *str = NULL;
    GetMemory(&str, 100);
    strcpy(str, "hello");
    printf("%s", str); // 错误：未格式化字符串
}
```

问题： `printf` 中应使用格式化字符串 `"%s"`。否则，程序可能会出现未定义行为。

15. 计算 `sizeof` 的值

```
char str[] = "raysharp.cn";
char *p = str;
int n = 10;
```

- `sizeof(str)`: `str` 是一个字符数组，包含 11 个字符（包括终止字符）。所以 `sizeof(str) = 12`。
- `sizeof(p)`: `p` 是一个 `char*` 指针，在 32 位系统下，`sizeof(p) = 4`。

- `sizeof(n)`: `n` 是一个 `int`, 在 32 位系统下, `sizeof(n) = 4`。

```
void Foo(char str[100]) {  
    // 参数 str 被视为 char* 类型  
    sizeof(str) // 实际上是 sizeof(char*), 即 4  
}
```

- `sizeof(str)`: 在函数参数中, `str` 被视为 `char*`, 所以 `sizeof(str) = 4`。

```
void *p = malloc(100);
```

- `sizeof(p)`: `p` 是 `void*` 指针, 在 32 位系统下, `sizeof(p) = 4`。

16. 32-bit 数 0x12345678 在 Little-endian 和 Big-endian 模式下的存放方式

- **Little-endian**: 低位字节存储在低地址, 存储顺序为: 78 56 34 12 从地址 0x4000 开始。
- **Big-endian**: 高位字节存储在低地址, 存储顺序为: 12 34 56 78 从地址 0x4000 开始。

17. 计算 long 变量中有多少个 bit 的值是 1

```
#include <stdio.h>  
  
int countBits(long num) {  
    int count = 0;  
    while (num) {  
        count += num & 1;  
        num >>= 1;  
    }  
    return count;  
}  
  
int main() {  
    long num = 0x12345678;  
    printf("Number of 1 bits: %d\n", countBits(num));  
    return 0;  
}
```

18. Linux 内核同步方法

- 互斥锁 (Mutexes)
- 自旋锁 (Spinlocks)
- 读写锁 (Read-Write Locks)
- 信号量 (Semaphores)
- 条件变量 (Condition Variables)

19. 用 shell 脚本判断变量 `a` 的值

```
#!/bin/bash

a=0
if [ $a -eq 0 ]; then
    echo "correct"
elif [ $a -eq 1 ]; then
    echo "incorrect"
else
    echo "Invalid value"
fi
```

20. 用 Linux 命令查找当前文件下所有 C 文件

```
find . -name "*.c"
```

22. 判断 A B C D 四个表达式的是否正确，并写出经过表达式中 `a` 的值

```
int a = 4;
```

(A) `a += (a++);`
// `a++` 先使用 `a` 的值，再增加 1。结果为 $4 + 4 = 8$ 。`a` 变成 5。

(B) `a += (++a);`
// `++a` 先增加 1，再使用 `a` 的值。结果为 $5 + 6 = 11$ 。`a` 变成 6。

(C) `(a++) += a;`
// `a++` 先使用 `a` 的值，再增加 1。结果为 $4 + 5 = 9$ 。`a` 变成 5。

(D) `(++a) += (a++);`
// `++a` 先增加 1，再使用 `a` 的值。结果为 $6 + 6 = 12$ 。`a` 变成 7。

希望这些答案对你有帮助！如果有任何问题或需要进一步的解释，请告诉我。